2D GAMES · SAMPLE BUILD PLAN

Game 03 — Sliding Puzzle (Complete Plan)

Game 3 of 7 Complexity: ⭐ ⭐ Medium Estimate: 1–2 days Status: not started

> Complete plan (supersedes the 2026-09-09 sample). Status: **not started** — build-ready as written. Slug `sliding-puzzle`; folder `apps/frontend/public/games/sliding-puzzle/`.

1. Overview

Classic 15-puzzle: slide tiles into order. Hook is the personal record pair — "4×4 in 2:41 · 213 moves" — plus guaranteed-solvable shuffles so nobody ever hits an impossible board.

2. Complete game spec

- Sizes: **3×3** (default), **4×4** (classic), 5×5 (P3). Tiles numbered 1..N^2-1, blank last.
- **Shuffle:** from the solved state, apply random legal moves — 120 (3×3), 250 (4×4), 400 (5×5) — never undoing the immediately previous move. Solvability is guaranteed by construction; the inversion-count check runs anyway in tests.
- **Move counting:** a single-tile slide = 1 move; a row/column segment push = 1 move per tile displaced.
- **Timer:** starts on the first move, stops on the winning move; accumulates across pauses (`visibilitychange`), never runs during `menu` / `won`.
- **Win:** tiles in order, blank at end → overlay (§4). Input locked after the winning move.
- **Score:** `score = max(0, sizeBonus - floor(moves/2) - floor(seconds/5))` with `sizeBonus` 300 / 500 / 800 for 3×3 / 4×4 / 5×5.
- **Wrong-tile feedback:** tapping a tile not in the blank's row/column = shake animation, no move, no move-count change.

3. Progression model

Best records per size (time + moves). Progression is self-referential (beat your record); "won at size N" unlocks nothing (all sizes always selectable — no fake gating).

4. Screens & UI

State	Elements
`menu`	Size picker (3 cards with best time/moves per size), ▶️ Play, mute toggle
`playing`	Board (CSS grid), HUD: moves, timer, [Restart] [Shuffle] [⌂ Menu]
`paused`	Overlay on `visibilitychange` / ⏸️ — board hidden (no sneaking moves), timer held
`won`	Overlay: 🎉 time, moves, score, per-size best badges ("NEW BEST!"), [Play again] [Bigger grid] [📤 Share]

Tiles are absolutely-positioned buttons moved via `transform: translate` transitions (180 ms ease); the blank is a missing tile. Board sizing: `min(92vw, 60dvh)` square.

5. Data model (localStorage)

```
game:sliding-puzzle:best:<size>  → { timeMs: 161000, moves: 213 }     # size = 3|4|5
game:sliding-puzzle:prefs        → { size: 3, muted: false }
```

6. File structure & function inventory

```
sliding-puzzle/
  index.html
  style.css
  core.js     # pure model — the test surface
  game.js     # rendering, input, timer, audio, share
```

`core.js` exports: `solvedBoard(n)`, `neighbors(i, n)`, `legalMoves(board, n)`, `shuffle(n, movesCount, rng) → board` (no-undo rule), `slideTile(board, n, index) → { board, moved } | null` `(single tile or segment),` `isSolved(board)` `,` `scoreFor(size, moves, seconds)` `.` `game.js`: render from board array (keyed by tile value), pointer + keyboard input, timer accumulator, best-score logic, share text.

7. Edge cases

1. Segment slide: tap tile k in blank's row/col → tiles between k and blank shift one step toward blank; move count += displaced count; animation staggered 40 ms/tile.
2. Shuffle never leaves the board solved (assert; re-shuffle if it somehow lands solved).
3. Pause hides the board (overlay is opaque) — no solving while paused; timer freezes.
4. Keyboard focus: arrows move the tile *into* the blank (tile travels in arrow direction); focus ring visible; Enter = same as tap on focused tile.
5. Storage disabled → no records, game still fully playable.
6. Rapid tap during transition animation → state updates immediately, animation catches up (never queue clicks).
7. `won` input lock: any tap/slide after the final move is a no-op until an overlay button is used.

8. Testing plan (core.js under node:test or test.html)

- 100 shuffles per size → `isSolved` false + inversion-parity solvable (3×3 & 4×4 rules).
- No-undo rule: replay any shuffle log → no move is the inverse of its predecessor.
- `slideTile`: adjacent slide; row segment (2 and 3 tiles); column segment; illegal tile → null; move counts correct.
- `isSolved` on solved + known-shuffled boards.

- `scoreFor`: clamps at 0; monotonic in moves and seconds.

9. Task breakdown

P0 — playable core

- [ ] `core.js` model: solved/neighbors/legalMoves/slideTile/isSolved/shuffle + unit tests
- [ ] Board rendering with transform transitions; tap + segment slide; move counter
- [ ] Timer (accumulate across pause); win detection + overlay; restart/shuffle buttons

P1 — full rules & feel

- [ ] 3×3/4×4 size switch with per-size records; pause overlay (board hidden)
- [ ] Slide/snap animations, staggered segment push, wrong-tile shake, sound blips + mute
- [ ] Keyboard play (arrows + Enter) with visible focus

P2 — persistence/share/QA

- [ ] Best records per size + "NEW BEST" badge; share text (`{size}×{size} in m:ss · moves`)
- [ ] QA gate (master README §5): offline, isolation greps, 10-min phone session

P3 — polish

- [ ] 5×5 size; picture mode (CSS background-position split of any bundled-free image); single-level undo; solve-demo animation; swipe input

10. Acceptance criteria

- §8 tests green; shuffles always solvable (automated proof, 100 runs/size).
- Timer exact across pause/resume (±50 ms vs wall clock of played time).
- No input accepted after win; 3×3 fits 360 px with HUD visible, no scroll.
- Restart ≤ 1 tap; shuffle produces a visibly mixed board every time.

11. Deferred coupling (intentionally NOT built)

Footer/nav links, analytics events, achievements, backend leaderboards — per master README §7.